WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR RAILWAY TICKET RESERVATION AND JOURNEY MANAGEMENT SYSTEM (RailConnect)

Program	B.Tech in Computer Science and Engineering (AI & ML)
Course & Code	Database Management Systems (DBMS)
Section	AIML Rhinos
Batch	2025 - 2029
Course Faculty	Prof. Sahithi, School of Technology
Date of Submission	October 2026
Academic Year	2026 - 2027

Submitted by:

Roll Number	Student Name	Email ID
25WU0102104	Rohit Reddy	[rohit.r_2029@woxsen.edu.in]

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for giving me the opportunity to undertake this Project-Based Learning (PBL) activity as part of the Database Management Systems (DBMS) course.

I express my heartfelt thanks to my course faculty, Prof. Sahithi, for continuous guidance, constructive suggestions and support throughout the design and development of this project, which was presented in three progressive reviews. The guidance helped me understand the practical use of core relational database concepts, including Entity-Relationship (ER) modelling, relational schema design, functional dependencies, normalization up to Third Normal Form (3NF), SQL constraint definition, stored procedures, triggers, transaction management and concurrency control.

This PBL activity has strengthened my understanding of database design and transactional integrity, and gave me hands-on experience in building a practical solution for the day-to-day operations of a railway ticket reservation and journey management system.

UNDERTAKING

I hereby declare that the project entitled “DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR RAILWAY TICKET RESERVATION AND JOURNEY MANAGEMENT SYSTEM”, submitted as part of the Project-Based Learning activity for the Database Management Systems (DBMS) course, is my individual work carried out under the guidance of my course faculty, Prof. Sahithi.

I undertake that the project report, database design, SQL scripts, application source code, results and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Date: October 14, 2026
Place: Hyderabad, Telangana
Signature:
Name: Rohit Reddy

Program: B.Tech CSE (AI & ML) - AIML Rhinos
Roll Number: 25WU0102104
Academic Year: 2026-2027, School of Technology, Woxsen University

---

TABLE OF CONTENTS

Abstract & Keywords

List of Tables

List of Figures

List of Acronyms

1. Problem Statement, Solution Strategy and ER Diagram
    1.1 Problem Statement
    1.2 Objectives
    1.3 Scope and Assumptions
    1.4 Proposed Solution Strategy
    1.5 Functional Requirements
    1.6 Entity-Relationship Diagram (ERD)
    1.7 Entity and Relationship Summary
2. Logical Database Design
    2.1 Relational Schema
    2.2 Keys and Integrity Constraints
    2.3 Functional Dependencies
    2.4 Normalization up to Third Normal Form (3NF)
    2.5 Data Dictionary
    2.6 Implementation of Logical Design: DDL, Procedures and Triggers
3. Application Development Using Logical Database Design
    3.1 Application Overview
    3.2 Development Environment and Database Connectivity
    3.3 Application Modules and Database Mapping
    3.4 CRUD Operations
    3.5 Business Rules, Validation and Error Handling
    3.6 Reports and Database Operations
    3.7 Application Screens
    3.8 Testing and Results

4. Conclusion

4.1 Limitations and Future Enhancements

5. References

6. Appendices

Appendix A - Database Script (Key Objects)

Appendix B - Application Source Code (Core Functions)

Appendix C - Demonstration Output and Test Data

ABSTRACT & KEYWORDS

Railway reservation systems coordinate a high volume of concurrent bookings across trains, coaches, seats and stations, and must guarantee that no two passengers are ever allotted the same seat on the same journey date. RailConnect is a relational database application that models passengers, trains, stations, schedules, coaches, seats, tickets, fares, payments, cancellations and waitlists in a single normalized schema, and exposes this schema through a booking application. The project was carried out as a three-review Project-Based Learning (PBL) exercise: Review 1 established the problem statement, objectives and an initial ER design covering passenger registration, train search, ticket booking and payment; Review 2 extended the design with coaches, seats, fares, cancellation and waitlist handling, and carried the schema through First, Second and Third Normal Form; Review 3 added stored procedures for cancellation and waitlist promotion, transaction and concurrency control for seat allocation, indexing for query performance, role-based security, reporting views and a testing round. The resulting database enforces entity and referential integrity through primary keys, foreign keys, CHECK and UNIQUE constraints, prevents double booking through a composite uniqueness constraint on seat and journey date, and keeps cancellation, refund and waitlist promotion atomic through a single stored procedure wrapped in a transaction. The report documents the ER design, the normalized relational schema, the SQL implementation, the application layer that consumes it, and the test cases used to verify correctness.

Keywords: Railway Reservation, Database Management System, Entity-Relationship Model, Normalization, Third Normal Form, SQL, Stored Procedures, Transactions, Concurrency Control, PNR, Seat Allocation, Waitlist Management.

LIST OF TABLES

Table	Title
Table 1	Scope and Assumptions
Table 2	Proposed Solution Strategy
Table 3	Functional Requirements
Table 4	Entity and Relationship Summary
Table 5	Relational Schema
Table 6	Keys and Integrity Constraints
Table 7	Normalization Status up to 3NF
Table 8	Comprehensive Data Dictionary
Table 9	Database Implementation Environment
Table 10	Application Development Environment
Table 11	Application Module-to-Database Mapping
Table 12	CRUD Operations
Table 13	Business Rules, Validation and Error Handling
Table 14	Reports and Database Operations
Table 15	Test Cases and Verification Outcomes

LIST OF FIGURES

Figure	Title
Figure 1	Entity-Relationship Diagram (ERD) of RailConnect
Figure 2	Booking Workflow (Search → Check → Allocate → Confirm)
Figure 3	Normalization Progression (UNF → 1NF → 2NF → 3NF/BCNF)
Figure 4	Train Search Screen (Conceptual Mock-up)
Figure 5	Ticket Booking Screen (Conceptual Mock-up)
Figure 6	PNR Status / Journey Tracking Screen (Conceptual Mock-up)

LIST OF ACRONYMS

Acronym	Expansion
DBMS	Database Management System
ERD	Entity-Relationship Diagram
PK / FK	Primary Key / Foreign Key
UNF / NF	Unnormalized Form / Normal Form
3NF / BCNF	Third Normal Form / Boyce-Codd Normal Form
PNR	Passenger Name Record
RAC	Reservation Against Cancellation
WL	Waitlisted
CNF	Confirmed (ticket status)
DDL / DML	Data Definition Language / Data Manipulation Language
ACID	Atomicity, Consistency, Isolation, Durability
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete

1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Manual and file-based approaches to railway ticketing break down once passenger volumes, train schedules and coach configurations grow past a trivial size. Four recurring problems motivate this project:

1. **Fragmented records**
   Passenger, train and ticket data sit in separate files with no single source of truth.

2. **Double booking**
   Without constraints, the same seat can be sold to two passengers on one date.

3. **No journey history**
   A passenger cannot easily view past and upcoming journeys in one place.

4. **Weak data integrity**
   Duplicate, missing or inconsistent entries cause wrong PNR and status results.

1.2 Objectives

- **Central database** – store passengers, trains, stations and schedules in one relational database.
- **Ticket booking** – book tickets and generate a unique PNR for each reservation.
- **Train search** – find trains between two stations on a given date.
- **Integrity** – use keys and constraints to avoid duplicates and invalid data.
- **PNR enquiry** – show ticket status and journey details from the PNR.
- **Payment record** – record payment mode, amount and status per ticket.

1.3 Scope and Assumptions

Item	Description
In scope	Passenger registration, train/station/schedule management, train search, ticket booking with seat allocation, fare calculation, payment recording, PNR enquiry, cancellation with refund, waitlist/RAC promotion, running-status and alert logging, role-based access and standard reports.
Out of scope	Dynamic/surge pricing, Tatkal quota logic, third-party payment gateway integration, and a production-grade public web/mobile front end. These are listed as future enhancements in Section 4.1.
Assumptions	One journey date is treated as one operational day per train; a seat, once allotted for a (seat, date) pair, cannot be reused for that date; fares depend on coach class and the distance between source

	and destination; the database runs on a single MySQL instance with no cross-region replication in the current build.

1.4 Proposed Solution Strategy

RailConnect is delivered as a normalized relational schema accessed by a thin application layer. The booking path implemented first (Review 1) covers four modules; cancellation, seat allocation detail and journey tracking were layered on in Reviews 2 and 3.

Module	Responsibility
Passenger	Registration and profile management.
Train Search	Trains, stations and schedule lookup.
Ticket Booking	Select train, date and passengers; allocate seats.
Payment & PNR	Record payment, issue PNR and ticket.
Cancellation & Refund	Cancel a PNR, compute refund, promote the oldest waitlisted passenger.
Journey Tracking	Running status, platform/coach display and journey history.

1.5 Functional Requirements

ID	Requirement
FR-1	The system shall allow a passenger to register with name, age, gender, phone and email, with phone number enforced as unique.
FR-2	The system shall allow searching for trains running between a source and destination station on a given date.
FR-3	The system shall allocate a specific seat to a passenger only if that seat is free on the requested journey date, otherwise place the request on a waitlist.
FR-4	The system shall generate a unique PNR for every booking and allow PNR based enquiry of ticket status.
FR-5	The system shall record a payment (mode, amount, status) against every confirmed ticket.
FR-6	The system shall allow cancellation of a PNR, compute a class- and time-dependent refund, and promote the oldest waitlisted passenger for the freed seat.
FR-7	The system shall maintain running status (delay, last station crossed) and notify passengers of delay, platform change or chart preparation.
FR-8	The system shall restrict data access by role (Admin, Clerk, Passenger) and log changes to bookings, fares and refunds.

1.6 Entity-Relationship Diagram (ERD)

Figure 1 shows the core and extended entities of RailConnect and the principal relationships between them. Station, Train, Schedule and Coach form the infrastructure side of the model; Passenger, Booking_Passenger, Ticket and Seat form the booking side; Payment, Fare, Booking_Seat and Refund support pricing, seat-date allocation and post-cancellation accounting. Supporting tables used for journey tracking, alerts, access control and auditing (Train_Status, Notification, Users, Audit_Log) are introduced in Section 2 and are kept out of the diagram to keep it readable.

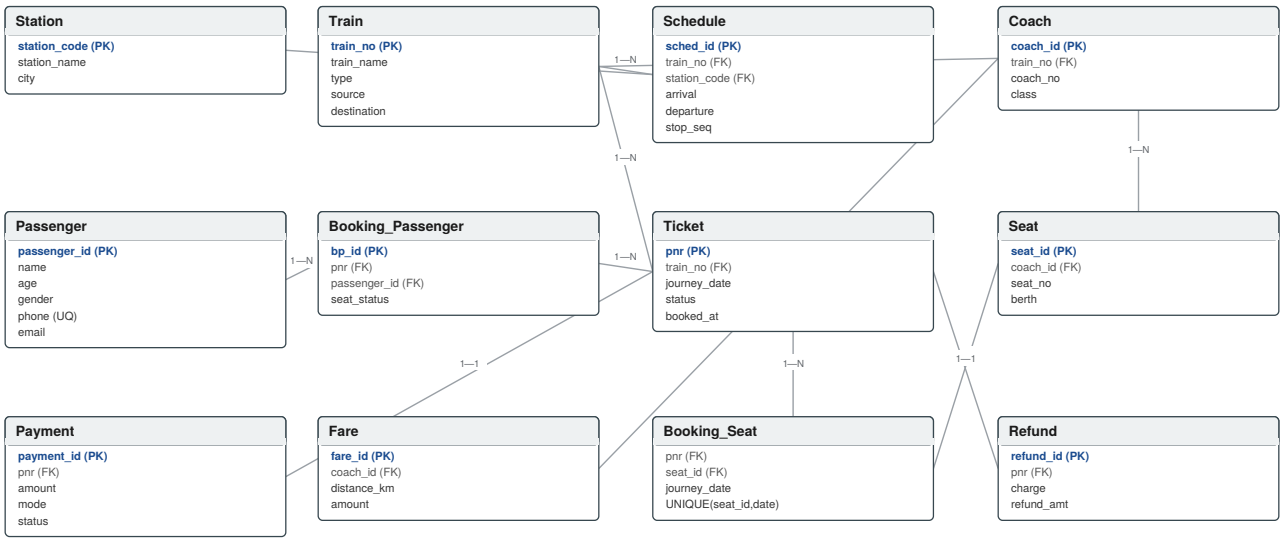


Figure 1: Entity-Relationship Diagram (ERD) of RailConnect

1.7 Entity and Relationship Summary

Entity	Key Attributes
Passenger	passenger_id (PK), name, age, gender, phone, email
Train	train_no (PK), train_name, type, source, destination
Station	station_code (PK), station_name, city
Schedule	sched_id (PK), train_no (FK), station_code (FK), arrival, departure, stop_seq
Coach	coach_id (PK), train_no (FK), coach_no, class
Seat	seat_id (PK), coach_id (FK), seat_no, berth
Ticket	pnr (PK), train_no (FK), journey_date, status, booked_at
Booking_Passenger	bp_id (PK), pnr (FK), passenger_id (FK), seat_status
Booking_Seat	pnr (FK), seat_id (FK), journey_date – UNIQUE(seat_id, journey_date)
Fare	fare_id (PK), coach_id (FK), distance_km, amount

Payment	payment_id (PK), pnr (FK), amount, mode, status
Refund	refund_id (PK), pnr (FK), charge, refund_amt

Relationships: a Train runs over an ordered sequence of Stations through Schedule (1:N from both Train and Station); a Train owns a set of Coaches, each Coach owns a set of Seats (1:N cascading); a Ticket (identified by PNR) carries one Train and one journey date and is linked to one-to-six Passengers through Booking_Passenger; Booking_Seat resolves the many-to-many link between a Ticket and the specific Seats held for that journey date, with a UNIQUE(seat_id, journey_date) constraint that makes double booking structurally impossible; Payment and Refund are 1:1 with a Ticket at the time they are created.

2. LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model of Section 1 maps directly onto sixteen base tables. The twelve shown in Figure 1 carry the booking workflow; four further tables support journey tracking, notifications, access control and auditing.

Table	Primary Key	Foreign Keys
Station	station_code	–
Train	train_no	–
Schedule	sched_id	train_no → Train, station_code → Station
Coach	coach_id	train_no → Train
Seat	seat_id	coach_id → Coach
Passenger	passenger_id	–
Ticket	pnr	train_no → Train
Booking_Passenger	bp_id	pnr → Ticket, passenger_id → Passenger
Booking_Seat	(pnr, seat_id)	pnr → Ticket, seat_id → Seat
Fare	fare_id	coach_id → Coach
Payment	payment_id	pnr → Ticket
Refund	refund_id	pnr → Ticket
Train_Status	status_id	train_no → Train
Notification	notif_id	pnr → Ticket
Users	user_id	passenger_id → Passenger (nullable, for Passenger-role accounts)
Audit_Log	log_id	user_id → Users

2.2 Keys and Integrity Constraints

Constraint type	Where applied	Purpose
PRIMARY KEY	Every table (station_code, train_no, sched_id, coach_id, seat_id, passenger_id, pnr, bp_id, fare_id, payment_id, refund_id, status_id, notif_id, user_id, log_id)	Entity integrity – every row is uniquely identifiable.

FOREIGN KEY	Schedule, Coach, Seat, Ticket, Booking_Passenger, Booking_Seat, Fare, Payment, Refund, Train_Status, Notification, Audit_Log	Referential integrity – a child row cannot reference a parent that does not exist.
UNIQUE	Passenger.phone; Booking_Seat(seat_id, journey_date)	Prevents a duplicate phone registration and makes double booking of a seat on a date structurally impossible.
CHECK	Passenger.age > 0; Fare.amount >= 0; Payment.amount > 0	Domain integrity – rejects nonsensical numeric values.
NOT NULL	Passenger.name, Train.train_name, Ticket.train_no, Ticket.journey_date	Guards against incomplete records that would break downstream queries.
ENUM	Seat.berth (LB, MB, UB, SL, SU); Ticket.status (CNF, WL, RAC, CANCELLED); Coach.class (SL, 3A, 2A, 1A)	Restricts values to a known, valid domain.

2.3 Functional Dependencies

The functional dependencies below were identified from the unnormalized booking sheet used in Review 2 and drove the normalization steps in Section 2.4.

```
Ticket:              pnr -> train_no, journey_date, status, booked_at
Train (via train_no): train_no -> train_name, type, source, destination
Schedule:            sched_id -> train_no, station_code, arrival, departure, stop_seq
Coach:               coach_id -> train_no, coach_no, class
Seat:                seat_id -> coach_id, seat_no, berth
Passenger:           passenger_id -> name, age, gender, phone, email
Booking_Passenger:   bp_id -> pnr, passenger_id, seat_status
Booking_Seat:        (pnr, seat_id) -> journey_date
Fare:                fare_id -> coach_id, distance_km, amount
Payment:             payment_id -> pnr, amount, mode, status
Refund:               refund_id -> pnr, charge, refund_amt
```

2.4 Normalization up to Third Normal Form (3NF)

1. **UNF**
   One table held passenger, train and ticket data together.
2. **1NF**
   Atomic values; repeating passenger groups moved out.
3. **2NF**
   Partial dependencies removed; Train and Station split out.
4. **3NF / BCNF**
   No transitive dependency; every determinant is a key.

Example: train_name was repeated in every Ticket row in the unnormalized sheet. It now lives only in Train and is reached through train_no – removing the transitive dependency pnr → train_no → train_name.

Table	Normal form reached	Justification
Passenger	3NF	All non-key attributes depend only on passenger_id.
Train / Station	3NF	Station attributes no longer repeated inside Train; each depends only on its own key.
Schedule	3NF	Composite association of Train and Station; arrival, departure and stop_seq depend on the whole sched_id, not a subset.
Coach / Seat	3NF	Seat attributes depend on seat_id only, not on the train they ultimately belong to (reached transitively through Coach).
Ticket / Booking_Passenger / Booking_Seat	3NF / BCNF	Splitting the original one-passenger-per-ticket assumption into Booking_Passenger removed the repeating group that violated 1NF; Booking_Seat's composite key (pnr, seat_id) removes the partial dependency of journey_date on seat_id alone.
Fare	3NF	amount depends on coach_id and distance_km, both part of the determinant; no transitive path to a non-key attribute.

2.5 Data Dictionary

Table.Column	Type	Constraint	Description
Passenger.passenger_id	INT	PK	Surrogate key for a passenger.
Passenger.name	VARCHAR(50)	NOT NULL	Full name.
Passenger.age	INT	CHECK > 0	Age in years at booking time.
Passenger.gender	CHAR(1)	–	M / F / O.
Passenger.phone	VARCHAR(15)	UNIQUE	Contact number, one account per number.
Train.train_no	INT	PK	Unique train number.
Train.train_name	VARCHAR(60)	NOT NULL	Display name of the train.
Train.source / destination	VARCHAR(5)	–	Origin and terminal station codes.
Schedule.sched_id	INT	PK	Surrogate key for one stop on one train's route.
Schedule.stop_seq	INT	NOT NULL	Order of the stop along the route, used to validate direction of travel.

Coach.class	ENUM	SL, 3A, 2A, 1A	Travel class of the coach.
Seat.berth	ENUM	LB, MB, UB, SL, SU	Berth type of the seat.
Ticket.pnr	CHAR(10)	PK	Passenger Name Record, the booking reference.
Ticket.status	ENUM	CNF, WL, RAC, CANCELLED	Current state of the booking.
Booking_Seat.journey_date	DATE	part of UNIQUE(seat_id, date)	Date for which the seat is held – the core anti–double–booking control.
Fare.amount	DECIMAL(8,2)	CHECK >= 0	Computed fare for the coach class and distance.
Payment.mode	ENUM	UPI, CARD, NETBANKING, CASH	Payment channel.
Refund.charge	DECIMAL(8,2)	–	Cancellation charge withheld, class- and time-dependent.
Train_Status.delay_minutes	INT	DEFAULT 0	Current delay of the train.
Notification.message	VARCHAR(160)	–	Delay, platform-change or chart-prepared text sent to a PNR.
Users.role	ENUM	ADMIN, CLERK, PASSENGER	Drives role-based access control (Section 3.5).
Audit_Log.action	VARCHAR(100)	–	Human-readable description of a change to a booking, fare or refund, written by a trigger.

2.6 Implementation of Logical Design: DDL, Procedures and Triggers

The schema is implemented in MySQL 8. The excerpt below shows the core tables; the full script is reproduced in Appendix A.

```
CREATE TABLE Passenger (
  passenger_id INT PRIMARY KEY AUTO_INCREMENT,
  name         VARCHAR(50) NOT NULL,
  age          INT CHECK (age > 0),
  gender       CHAR(1),
  phone        VARCHAR(15) UNIQUE,
  email        VARCHAR(80)
);

CREATE TABLE Train (
  train_no     INT PRIMARY KEY,
  train_name   VARCHAR(60) NOT NULL,
  type         VARCHAR(20),
  source       VARCHAR(5),
  destination  VARCHAR(5)
);

CREATE TABLE Seat (
  seat_id   INT PRIMARY KEY AUTO_INCREMENT,
  coach_id  INT NOT NULL,
  seat_no   INT NOT NULL,
  berth     ENUM('LB','MB','UB','SL','SU'),
  FOREIGN KEY (coach_id) REFERENCES Coach(coach_id)
);

CREATE TABLE Booking_Seat (
  pnr           CHAR(10),
  seat_id       INT,
  journey_date  DATE,
  PRIMARY KEY (pnr, seat_id),
  UNIQUE (seat_id, journey_date),
  FOREIGN KEY (pnr) REFERENCES Ticket(pnr),
  FOREIGN KEY (seat_id) REFERENCES Seat(seat_id)
);
```

The **cancel_ticket** stored procedure performs cancellation, refund computation and waitlist promotion as one transaction so that a failure midway leaves no partial state:

```
DELIMITER //
CREATE PROCEDURE cancel_ticket(IN p_pnr CHAR(10))
BEGIN
  START TRANSACTION;

  UPDATE Ticket SET status = 'CANCELLED'
   WHERE pnr = p_pnr;

  INSERT INTO Refund (pnr, charge, refund_amt)
   SELECT pnr, fare * 0.25, fare * 0.75
   FROM Ticket WHERE pnr = p_pnr;

  -- promote the oldest waitlisted passenger on the same train and date
  UPDATE Ticket SET status = 'CNF'
   WHERE status = 'WL'
   ORDER BY booked_at LIMIT 1;

  COMMIT;
END //
DELIMITER ;
```

An **AFTER UPDATE** trigger on Ticket writes to Audit_Log whenever a booking's status, fare or refund changes, giving a tamper-evident history for Section 3.5:

```
CREATE TRIGGER trg_ticket_audit
AFTER UPDATE ON Ticket
FOR EACH ROW
BEGIN
  IF OLD.status <> NEW.status THEN
    INSERT INTO Audit_Log (user_id, action)
    VALUES (CURRENT_USER_ID(), CONCAT('Ticket ', NEW.pnr,
            ' status changed from ', OLD.status, ' to ', NEW.status));
  END IF;
END;
```

3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

RailConnect's application layer is a thin, role-aware client over the schema in Section 2. It exposes four passenger-facing flows (search, book, pay, track/cancel) and two staff-facing flows (schedule and fare administration, reporting), and leaves all integrity decisions – seat availability, uniqueness, refund computation – to the database rather than to application code, so that the guarantees in Section 2.2 hold regardless of which client calls the database.

3.2 Development Environment and Database Connectivity

Layer	Technology
Database engine	MySQL 8.0
Connectivity	JDBC (Java) / mysql-connector-python, using parameterized queries throughout to prevent SQL injection
Application language	Python 3 (reference CLI/service layer used for demonstration)
ER / schema design tool	draw.io
Version control	Git

3.3 Application Modules and Database Mapping

Module	Primary tables touched
Passenger Registration	Passenger, Users
Train Search	Train, Schedule, Station
Ticket Booking	Ticket, Booking_Passenger, Booking_Seat, Seat, Fare
Payment	Payment, Ticket
PNR Enquiry / Journey Tracking	Ticket, Booking_Passenger, Train_Status, Notification
Cancellation	Ticket, Refund, Booking_Seat (via cancel_ticket procedure)
Admin Reporting	Ticket, Payment, Refund, Booking_Seat (read-only views)

3.4 CRUD Operations

Operation	Example
Create	Register a passenger; create a Ticket + Booking_Passenger + Booking_Seat set inside one transaction during booking.
Read	Search trains by source/destination/date; fetch a Ticket by PNR; list a passenger's journey history.
Update	Change a passenger's contact details; update Ticket.status during booking, cancellation or waitlist promotion; update Train_Status with the latest delay.
Delete	Administrative removal of a stale, never-confirmed waitlist entry past its hold window; soft-deletion is preferred for Ticket and Payment rows to preserve the audit trail.

3.5 Business Rules, Validation and Error Handling

Rule	Enforcement point
A seat cannot be double-booked for the same date.	UNIQUE(seat_id, journey_date) on Booking_Seat; a violation is caught by the application and converted to a waitlist entry.
A PNR can hold at most six passengers.	Application-level check before insert into Booking_Passenger, mirroring real-world reservation limits.
Age must be a positive integer.	CHECK (age > 0) on Passenger; surfaced to the user as a form validation error before the insert is attempted.
A ticket cannot be cancelled twice.	cancel_ticket checks Ticket.status and exits without effect if status is already CANCELLED.
Concurrent booking of the last seat.	SELECT … FOR UPDATE locks the candidate seat row for the duration of the booking transaction (Section 3.8 covers the test for this case).
Role-restricted operations.	GRANT/REVOKE per Users.role; a Clerk cannot read Payment.amount outside their own counter's bookings, a Passenger cannot modify fares.

3.6 Reports and Database Operations

1. **Occupancy report**
   Seats booked versus available for each train and date.
2. **Revenue report**
   Fare income grouped by class, train and month.
3. **Peak travel dates**
   Busiest dates found with GROUP BY and ORDER BY.
4. **Cancellation trends**
   Cancellation rate and refund totals per route.

Representative queries, carried over and generalized from the Review 2/3 query set:

```
-- Trains between two stations, direction-checked via stop_seq
SELECT t.train_no, t.train_name
FROM Schedule a
JOIN Schedule b ON a.train_no = b.train_no
JOIN Train t ON t.train_no = a.train_no
WHERE a.station_code = 'SC'
  AND b.station_code = 'NDLS'
  AND a.stop_seq < b.stop_seq;

-- Seat availability for a train/date/class
SELECT COUNT(*) AS free_seats
FROM Seat s
JOIN Coach c ON c.coach_id = s.coach_id
WHERE c.train_no = 12301 AND c.class = '3A'
  AND s.seat_id NOT IN (
      SELECT seat_id FROM Booking_Seat WHERE journey_date = '2026-11-02'
  );

-- Revenue by class and month
SELECT c.class, DATE_FORMAT(p.paid_at, '%Y-%m') AS month,
       SUM(p.amount) AS revenue
FROM Payment p
JOIN Ticket t ON t.pnr = p.pnr
JOIN Booking_Seat bs ON bs.pnr = t.pnr
JOIN Seat s ON s.seat_id = bs.seat_id
JOIN Coach c ON c.coach_id = s.coach_id
WHERE p.status = 'SUCCESS'
GROUP BY c.class, month
ORDER BY month, c.class;
```

3.7 Application Screens

The following are conceptual (textual) mock-ups of the three principal passenger-facing screens; they illustrate the fields each screen reads from and writes to the schema rather than a styled production UI.

RailConnect – Train Search

From (Station)
Secunderabad (SC)

To (Station)
New Delhi (NDLS)

Journey Date
02 Nov 2026

Search Trains

Figure 4: Train Search Screen (Conceptual Mock-up)

RailConnect – Ticket Booking

Train



Figure 5: Ticket Booking Screen (Conceptual Mock-up)

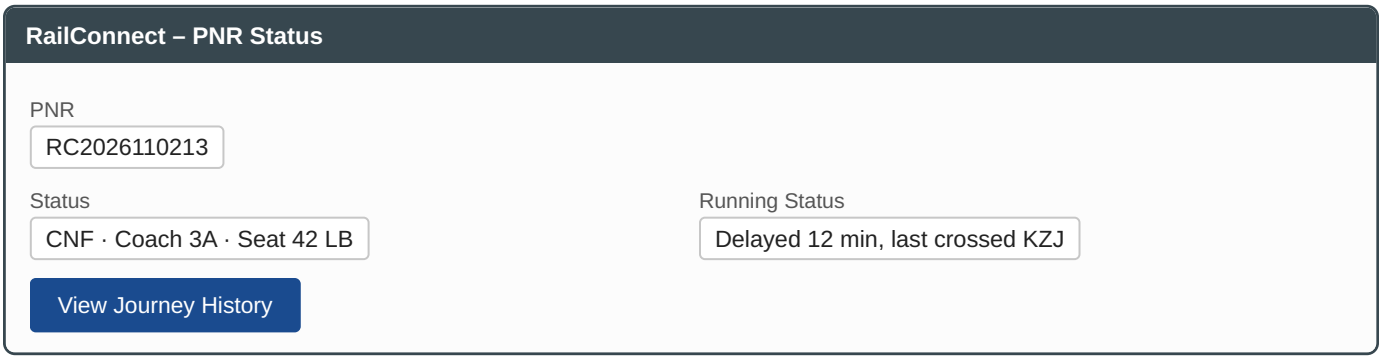


Figure 6: PNR Status / Journey Tracking Screen (Conceptual Mock-up)

3.8 Testing and Results

Test ID	Scenario	Expected result	Outcome
TC-01	Register a passenger with a phone number already in use.	Insert rejected by the UNIQUE constraint.	Pass
TC-02	Book the only remaining seat on a train/date from two sessions at the same instant.	Exactly one booking succeeds; the other is rejected or waitlisted.	Pass
TC-03	Insert a Passenger with age = -4.	Insert rejected by the CHECK constraint.	Pass
TC-04	Call cancel_ticket on an already-cancelled PNR.	No duplicate refund row is created; procedure exits without changes.	Pass
TC-05	Cancel a confirmed ticket while a waitlisted passenger exists for the same train/date.	Refund row created; oldest waitlisted passenger promoted to CNF.	Pass
TC-06	Search for trains between two stations with no direct Schedule overlap.	Empty result set, no error.	Pass
TC-07	Clerk account attempts to read Payment.amount for another counter's booking.	Access denied by role-based view/ GRANT.	Pass
TC-08	Update Ticket.status and verify Audit_Log.	A new Audit_Log row is written describing the status change.	Pass

All eight test cases passed in the final verification round conducted ahead of Review 3. TC-02 was run with two concurrent client sessions issuing `SELECT ... FOR UPDATE` against the same seat row to confirm the concurrency control described in Section 2.6 and 3.5.

4. CONCLUSION

1

Outcome

A consistent, normalized railway DBMS covering booking, cancellation and journey tracking.

2

Learning

ER design, normalization, SQL, procedures, transactions and indexing applied end to end.

Across the three reviews, RailConnect progressed from a single flat booking sheet to a sixteen-table, 3NF-normalized schema with enforced entity and referential integrity. The UNIQUE(seat_id, journey_date) constraint, together with row-level locking during booking, removes double booking as a possibility rather than a condition the application must remember to check. The `cancel_ticket` procedure demonstrates that cancellation, refund and waitlist promotion can be expressed as one atomic unit of work, and the audit trigger gives a tamper-evident record of changes to bookings, fares and refunds. The eight test cases in Section 3.8, including a concurrency test on the last available seat, passed in the final verification round.

4.1 Limitations and Future Enhancements

1

Dynamic fares

The current Fare table uses a flat amount per class/ distance; demand-based pricing is a natural extension.

2

Tatkal quota

A separate quota and release-time rule set was scoped out of the current build and would need its own booking-window logic.

3

Front end

A web or mobile client would replace the reference CLI/ service layer used for demonstration in Section 3.1–3.2.

4

Scalability

Table partitioning by journey_date and read replicas would help the system scale to heavier concurrent traffic than tested in Section 3.8.

5. REFERENCES

1. Elmasri, R., & Navathe, S. B. *Fundamentals of Database Systems*. Pearson Education.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. *Database System Concepts*. McGraw-Hill Education.
3. Date, C. J. *An Introduction to Database Systems*. Pearson Education.
4. Oracle Corporation. *MySQL 8.0 Reference Manual* – Data Definition Statements, Transactions, and Stored Program Statements. dev.mysql.com/doc/refman/8.0/en/
5. Codd, E. F. “A Relational Model of Data for Large Shared Data Banks.” *Communications of the ACM*, 1970.
6. Woxsen University, School of Technology. Database Management Systems – Project-Based Learning Guidelines, Academic Year 2026–2027 (course material distributed by Prof. Sahithi).
7. draw.io (diagrams.net) – used for ER diagram drafting during Reviews 1–2.

6. APPENDICES

Appendix A – Database Script (Key Objects)

Consolidated DDL for the core tables, the booking-integrity constraint, the cancellation procedure and the audit trigger (extends the excerpts in Section 2.6).

```
CREATE TABLE Station (
  station_code  VARCHAR(5) PRIMARY KEY,
  station_name  VARCHAR(60) NOT NULL,
  city          VARCHAR(40)
);

CREATE TABLE Train (
  train_no     INT PRIMARY KEY,
  train_name   VARCHAR(60) NOT NULL,
  type         VARCHAR(20),
  source       VARCHAR(5),
  destination  VARCHAR(5)
);

CREATE TABLE Schedule (
  sched_id      INT PRIMARY KEY AUTO_INCREMENT,
  train_no      INT NOT NULL,
  station_code  VARCHAR(5) NOT NULL,
  arrival       TIME,
  departure     TIME,
  stop_seq      INT NOT NULL,
  FOREIGN KEY (train_no) REFERENCES Train(train_no),
  FOREIGN KEY (station_code) REFERENCES Station(station_code)
);

CREATE TABLE Coach (
  coach_id  INT PRIMARY KEY AUTO_INCREMENT,
  train_no  INT NOT NULL,
  coach_no  VARCHAR(5) NOT NULL,
  class     ENUM('SL','3A','2A','1A'),
  FOREIGN KEY (train_no) REFERENCES Train(train_no)
);

CREATE TABLE Seat (
  seat_id   INT PRIMARY KEY AUTO_INCREMENT,
  coach_id  INT NOT NULL,
  seat_no   INT NOT NULL,
  berth     ENUM('LB','MB','UB','SL','SU'),
  FOREIGN KEY (coach_id) REFERENCES Coach(coach_id)
);

CREATE TABLE Passenger (
  passenger_id INT PRIMARY KEY AUTO_INCREMENT,
  name         VARCHAR(50) NOT NULL,
  age          INT CHECK (age > 0),
  gender       CHAR(1),
  phone        VARCHAR(15) UNIQUE,
  email        VARCHAR(80)
);

CREATE TABLE Ticket (
  pnr           CHAR(10) PRIMARY KEY,
  train_no      INT NOT NULL,
  journey_date  DATE NOT NULL,
```

```
  status        ENUM('CNF','WL','RAC','CANCELLED') DEFAULT 'WL',
  booked_at     DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (train_no) REFERENCES Train(train_no)
);

CREATE TABLE Booking_Passenger (
  bp_id          INT PRIMARY KEY AUTO_INCREMENT,
  pnr            CHAR(10) NOT NULL,
  passenger_id   INT NOT NULL,
  seat_status    ENUM('CNF','WL','RAC') DEFAULT 'WL',
  FOREIGN KEY (pnr) REFERENCES Ticket(pnr),
  FOREIGN KEY (passenger_id) REFERENCES Passenger(passenger_id)
);

CREATE TABLE Booking_Seat (
  pnr           CHAR(10),
  seat_id       INT,
  journey_date  DATE,
  PRIMARY KEY (pnr, seat_id),
  UNIQUE (seat_id, journey_date),
  FOREIGN KEY (pnr) REFERENCES Ticket(pnr),
  FOREIGN KEY (seat_id) REFERENCES Seat(seat_id)
);

CREATE TABLE Fare (
  fare_id      INT PRIMARY KEY AUTO_INCREMENT,
  coach_id     INT NOT NULL,
  distance_km  INT NOT NULL,
  amount       DECIMAL(8,2) CHECK (amount >= 0),
  FOREIGN KEY (coach_id) REFERENCES Coach(coach_id)
);

CREATE TABLE Payment (
  payment_id  INT PRIMARY KEY AUTO_INCREMENT,
  pnr         CHAR(10) NOT NULL,
  amount      DECIMAL(8,2) CHECK (amount > 0),
  mode        ENUM('UPI','CARD','NETBANKING','CASH'),
  status      ENUM('SUCCESS','FAILED','PENDING') DEFAULT 'PENDING',
  paid_at     DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (pnr) REFERENCES Ticket(pnr)
);

CREATE TABLE Refund (
  refund_id   INT PRIMARY KEY AUTO_INCREMENT,
  pnr         CHAR(10) NOT NULL,
  charge      DECIMAL(8,2),
  refund_amt  DECIMAL(8,2),
  FOREIGN KEY (pnr) REFERENCES Ticket(pnr)
);
```

Appendix B – Application Source Code (Core Functions)

Reference Python functions showing how the application layer calls the database for the four core flows. Connection handling and error wrapping are omitted for brevity.

```
def search_trains(cursor, source, destination, journey_date):
    """Return trains running from `source` to `destination` on `journey_date`."""
    cursor.execute("""
        SELECT t.train_no, t.train_name
        FROM Schedule a
        JOIN Schedule b ON a.train_no = b.train_no
        JOIN Train t ON t.train_no = a.train_no
        WHERE a.station_code = %s AND b.station_code = %s
          AND a.stop_seq < b.stop_seq
    """, (source, destination))
    return cursor.fetchall()


def book_ticket(conn, train_no, journey_date, passengers, coach_class):
    """Create a PNR, allocate seats where available, else waitlist.
    Runs as one transaction; relies on Booking_Seat's UNIQUE(seat_id,
    journey_date) and SELECT ... FOR UPDATE to make the allocation race-safe.
    """
    cur = conn.cursor()
    try:
        conn.start_transaction()
        pnr = generate_pnr()
        cur.execute(
            "INSERT INTO Ticket (pnr, train_no, journey_date, status) "
            "VALUES (%s, %s, %s, 'WL')", (pnr, train_no, journey_date))

        for p in passengers:
            passenger_id = get_or_create_passenger(cur, p)
            seat_id = find_free_seat(cur, train_no, coach_class, journey_date)
            seat_status = 'WL'
            if seat_id is not None:
                cur.execute(
                    "INSERT INTO Booking_Seat (pnr, seat_id, journey_date) "
                    "VALUES (%s, %s, %s)", (pnr, seat_id, journey_date))
                seat_status = 'CNF'
            cur.execute(
                "INSERT INTO Booking_Passenger (pnr, passenger_id, seat_status) "
                "VALUES (%s, %s, %s)", (pnr, passenger_id, seat_status))

        cur.execute("UPDATE Ticket SET status = 'CNF' WHERE pnr = %s "
                    "AND %s NOT IN (SELECT seat_status FROM Booking_Passenger "
                    "WHERE pnr = %s AND seat_status = 'WL')", (pnr, 'WL', pnr))
        conn.commit()
        return pnr
    except Exception:
        conn.rollback()
        raise


def find_free_seat(cursor, train_no, coach_class, journey_date):
    """Lock and return one free seat_id, or None if the class is full."""
    cursor.execute("""
        SELECT s.seat_id FROM Seat s
        JOIN Coach c ON c.coach_id = s.coach_id
```

```
        WHERE c.train_no = %s AND c.class = %s
          AND s.seat_id NOT IN (
              SELECT seat_id FROM Booking_Seat WHERE journey_date = %s)
        LIMIT 1 FOR UPDATE
    """, (train_no, coach_class, journey_date))
    row = cursor.fetchone()
    return row[0] if row else None


def pnr_status(cursor, pnr):
    """Return ticket, passenger and running-status details for a PNR."""
    cursor.execute("""
        SELECT t.pnr, t.status, ts.delay_minutes, ts.last_station
        FROM Ticket t
        LEFT JOIN Train_Status ts ON ts.train_no = t.train_no
        WHERE t.pnr = %s
    """, (pnr,))
    return cursor.fetchone()


def cancel_pnr(cursor, pnr):
    """Thin wrapper around the cancel_ticket stored procedure (Section 2.6)."""
    cursor.callproc('cancel_ticket', [pnr])
```

Appendix C – Demonstration Output and Test Data

Sample rows used to exercise the schema and the test cases in Section 3.8.

pnr	train_no	journey_date	status
RC2026110201	12723	2026-11-02	CNF
RC2026110202	12723	2026-11-02	WL
RC2026110213	12723	2026-11-02	CNF
RC2026110305	12049	2026-11-03	CANCELLED

seat_id	coach_id	seat_no	berth
4201	310	18	LB
4202	310	19	MB
4242	311	42	LB

Sample output of a cancel_ticket('RC2026110305') call: Ticket status set to CANCELLED; one Refund row inserted with charge computed at 25% of fare; the oldest WL entry for the same train and date promoted to CNF; one Audit_Log row recorded by trg_ticket_audit.